

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH  
TRƯỜNG ĐẠI HỌC BÁCH KHOA  
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



## HỌC SÂU VÀ ỨNG DỤNG TRONG THỊ GIÁC MÁY TÍNH

---

Bài tập – Nhóm 4

### Bài tập lớn số 2

---

GVHD: TS. Lê Thành Sách  
SV thực hiện: Nguyễn Hữu Trường - 2470573  
Nguyễn Nhật Thanh - 2570316  
Võ Thị Bích Phượng - 2470570

## Mục lục

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Giới thiệu</b>                                                             | <b>2</b>  |
| <b>2</b> | <b>Cơ sở Lý thuyết</b>                                                        | <b>2</b>  |
| 2.1      | Bài toán Phát hiện đối tượng (Object Detection)                               | 2         |
| 2.2      | Các Chỉ số Đánh giá Cốt lõi                                                   | 2         |
| 2.2.1    | 1. Ma trận nhầm lẫn (Confusion Matrix) trong phát hiện đối tượng              | 2         |
| 2.2.2    | 2. Biểu số Cân bằng Trọng yếu: Precision, Recall và F1-Score                  | 3         |
| 2.2.3    | 3. Tiêu chuẩn Vàng Đánh giá: AP và mAP                                        | 3         |
| 2.2.4    | 4. Hiệu suất Đo lường Cấu trúc (Hardware Metrics)                             | 3         |
| 2.3      | Phân loại Kiến trúc: Mạng One-stage và Two-stage                              | 3         |
| <b>3</b> | <b>Phân tích Dữ liệu Khám phá (EDA)</b>                                       | <b>4</b>  |
| 3.1      | Tổng quan và Cấu trúc tập dữ liệu KITTI                                       | 4         |
| 3.2      | Phân tích Phân phối Hình thái học (Morphological Data Visualization)          | 4         |
| 3.3      | Phân tích Ma trận Mật độ Sự cố và Độ Khó Che Khuất (Occlusions)               | 8         |
| <b>4</b> | <b>Chuẩn bị Dataset, DataLoader và Dữ liệu đầu vào</b>                        | <b>9</b>  |
| 4.1      | Quản lý Dữ liệu và DataLoader                                                 | 9         |
| 4.2      | Kỹ thuật Data Augmentation Nâng cao                                           | 9         |
| <b>5</b> | <b>Kiến trúc Mô hình Học Sâu - So sánh và Đánh giá</b>                        | <b>9</b>  |
| 5.1      | 1. Phương pháp Faster R-CNN (Kiến trúc Two-stage)                             | 9         |
| 5.2      | 2. Phương pháp YOLOv8m (Kiến trúc One-stage / Anchor-Free)                    | 10        |
| 5.3      | 3. Phương pháp DETR (Kiến trúc Transformer-based / Set Prediction)            | 11        |
| 5.4      | 4. So sánh đặc điểm kiến trúc: CNN-based (YOLOv8) vs Transformer-based (DETR) | 13        |
| 5.5      | 5. Cấu hình và Tham số Huấn luyện: YOLOv8s vs DETR                            | 14        |
| <b>6</b> | <b>Kết quả Thực nghiệm và Phân tích</b>                                       | <b>15</b> |
| 6.1      | Số liệu Thực nghiệm Đánh giá Độ chính xác                                     | 15        |
| 6.2      | Số liệu Đánh giá Tốc độ Inference (FPS Real-time Benchmark)                   | 16        |
| <b>7</b> | <b>Tổng kết</b>                                                               | <b>17</b> |
|          | <b>Tài liệu tham khảo</b>                                                     | <b>17</b> |

## 1 Giới thiệu

Báo cáo này trình bày kết quả thực hiện Bài tập lớn số 2 trong khuôn khổ môn học Học sâu và Ứng dụng trong Thị giác máy tính. Căn cứ theo yêu cầu từ tài liệu mô tả đề án, sinh viên được định hướng vận dụng kiến thức mạng học sâu để giải quyết một trong ba bài toán hạt nhân của thị giác máy tính: **Phân loại ảnh định danh (Image Classification)**, **Phát hiện đối tượng (Object Detection)**, hoặc **Phân vùng ngữ nghĩa cảnh vật (Semantic Segmentation)**. Đồng thời, đề bài đặt ra tiêu chuẩn khắt khe về việc hiện thực hóa toàn vẹn vòng đời một dự án (End-to-End Pipeline), trải dài từ khâu Phân tích Khám phá Định lượng (EDA), thiết lập DataLoader tích hợp kỹ thuật Augmentation, cho đến lập trình mô hình, kiểm định độ đo và phân tích đánh giá nguyên nhân lỗi (Error Analysis).

Dựa trên những yêu cầu đó, nhóm thực hiện quyết định chọn hướng đi **Phát hiện đối tượng từ ảnh (Object Detection)**. Lý do cho sự lựa chọn này xuất phát từ tính ứng dụng sống còn và đầy thử thách của bài toán định vị vị trí trong nền kỹ thuật công nghiệp hiện đại, đặc biệt là mảng trí tuệ tự hành (Autonomous Driving). Mục tiêu cốt lõi của nghiên cứu nhằm giải phẫu rạch ròi cấu trúc, đối chiếu hiệu năng giữa hai hệ sinh thái kiến trúc đối lập: đại diện truy xuất One-stage thời gian thực (**YOLOv8**) và nền tảng chuẩn xác Two-stage (**Faster R-CNN** tích hợp xương sống ResNet50-FPN).

Tập dữ liệu được nhóm tuyển sử dụng là **KITTI**, một bộ dữ liệu hình ảnh phức tạp tiêu chuẩn quốc tế chuyên phục vụ nghiên cứu kịch bản giao thông phức hợp. Các nội dung được trình bày tường tận phía dưới sẽ là minh chứng cụ thể cho hành trình áp dụng kiến thức tiên tiến để huấn luyện máy tính tháo gỡ điểm mù từ môi trường thực tế.

## 2 Cơ sở Lý thuyết

### 2.1 Bài toán Phát hiện đối tượng (Object Detection)

Phát hiện đối tượng là một trong những bài toán cốt lõi của Thị giác máy tính (Computer Vision), đòi hỏi máy tính không chỉ xác định xem có những loại vật thể nào xuất hiện trong hình ảnh (Image Classification - Phân loại ảnh), mà còn phải định vị chính xác và bao bọc các vật thể đó thông qua hệ tọa độ của các Hộp bao (Bounding Boxes). Đầu ra thông thường của một thuật toán Object Detection bao gồm một danh sách các hộp giới hạn, mỗi hộp được gắn liền với một nhãn phân loại (Class label) và điểm tin cậy xác suất (Confidence score).

### 2.2 Các Chỉ số Đánh giá Cốt lõi

Để đo lường định lượng và so sánh công bằng độ chính xác lẫn hiệu năng của thuật toán phát hiện đối tượng, cộng đồng nghiên cứu quy chiếu dựa trên một hệ thống các chuẩn đo lường quốc tế:

#### 2.2.1 1. Ma trận nhầm lẫn (Confusion Matrix) trong phát hiện đối tượng

Khác với phân loại ảnh thông thường, việc xác định đúng/sai (True/False) trong Object Detection dựa vào chỉ số **IoU (Intersection over Union)** – tỷ lệ diện tích phần giao nhau chia cho diện tích phần hợp tập của hộp dự đoán (Predicted Box) và hộp thực tế (Ground-truth Box). Nếu IoU vượt qua một ngưỡng quy định (Threshold, thường là 0.5), dự đoán được xem là khớp. Từ sự đối soát này, hệ thống sản sinh:

- **True Positive (TP):** Mô hình phát hiện chính xác vật thể, phân loại đúng nhãn dán, và khung Bounding Box ôm sát với độ chồng lấp  $IoU \geq Threshold$ .
- **False Positive (FP - Báo động giả):** Mô hình nhận diện sai một vật thể (ví dụ: ảnh nền rác bị nhận diện nhầm, dự đoán sai lớp, hoặc khung bao bị lệch nát khiến  $IoU < Threshold$ ).
- **False Negative (FN - Bỏ lỡ):** Mô hình hoàn toàn "mù" trước một vật thể có thực trong ảnh, không sinh ra dự đoán nào hoặc sinh ra nhưng bị vớt bỏ lập tức vì độ tin cậy quá thấp.
- **Lưu ý: True Negative (TN)** (không có xe và mô hình bảo không có) không được xem xét sử dụng để đo đạc vì số lượng hộp bao trống (background boxes) trên ảnh là vô hạn, làm nhiễu mẫu số.

### 2.2.2 2. Biến số Cân bằng Trọng yếu: Precision, Recall và F1-Score

Trong bài toán xe tự hành, tồn tại một sự đánh đổi sinh tử (Trade-off) giữa độ đo Precision và Recall:

- **Precision (Độ chuẩn xác):**  $Precision = \frac{TP}{TP+FP}$ . Tỷ lệ này biểu thị: trong 100 vật thể thuật toán "tự tin khoanh vùng", thực sự có bao nhiêu đồ vật là thật. Lỗi FP cao (nhận diện cột điện thành gã đi bộ) khiến Precision giảm mạnh, khiến xe tự lái sẽ phanh gấp liên tục gây sốc an toàn.
- **Recall (Độ phủ):**  $Recall = \frac{TP}{TP+FN}$ . Thông số biểu thị: trong 100 vật thể *thực sự đang đứng trên đường*, thuật toán gom bắt được bao nhiêu cái. Lỗi FN cao (bỏ sót chiếc xe tải) khiến Recall trượt dài, xe tự lái sẽ đâm trực diện mục tiêu không tránh né.
- **F1-Score:** Là trung bình điều hòa của cả hai ( $F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$ ). Một mô hình thương mại xuất sắc phải giữ vững khả năng dự đoán cân bằng định thái tạo ra đường tiệm cận độ tin cậy đỉnh cao cho chỉ số F1 tốt nhất.

### 2.2.3 3. Tiêu chuẩn Vàng Đánh giá: AP và mAP

Hai mô hình sở hữu F1-score giống nhau nhưng hành vi phản ứng điểm tin cậy lại rất khác biệt. Để đánh giá đường dài toàn diện, **Precision-Recall Curve (Đường cong PR)** được vẽ ra nhờ cơ chế quét thử các dự đoán từ mốc Confidence cực đại xuống thấp nhất:

- **AP (Average Precision):** Là tổng diện tích phân bố nằm dưới đường cong PR của *một phân lớp cụ thể* (Ví dụ: AP của Class Car, AP của Class Cyclist). Tính tích phân đồ thị này càng lấp đầy tiệm cận mốc 1.0 (100%) thì mạng thu nạp đối tượng đó càng hoàn hảo.
- **mAP (Mean Average Precision):** Trung bình cộng của chuẩn điểm AP trên *tất cả các nhãn lớp* có trong tập dữ liệu (KITTI có 8 lớp).
- **Chuẩn đo lường gắt gao mAP@0.5:0.95:** Theo hệ quy chuẩn giám định đo lường tân tiến của COCO, việc cố định IoU ở mức 0.5 là quá dễ dãi. Chuẩn đánh giá cực khoai mAP@0.5:0.95 sẽ lấy trung bình điểm mAP tích phân quét vắt kiệt qua 10 biến ngưỡng IoU liên tiếp nâng dần lên (0.50, 0.55, 0.60, ... 0.95). Chỉ số này trừng phạt cực nặng các thuật toán dán nhãn định danh trùng đối tượng nhưng kết cấu vẽ viền khung thừa thải, méo xệch so với thân xe.

### 2.2.4 4. Hiệu suất Đo lường Cấu trúc (Hardware Metrics)

Bên cạnh độ mẫn cảm bắt dính hộp ảnh, hao phí tài nguyên đóng vai trò tối thượng khi triển khai lên máy tính biên (Jetson, Raspberry Pi) của xe tự lái:

- **FPS (Frames Per Second):** Khối lượng điểm ảnh số hóa xử lý thành công trên 1 giây tính. Ngưỡng tốc độ  $FPS \geq 30$  được xem định mốc cấu trúc luân lý an toàn của chuẩn xử lý Thời gian thực (Real-time).
- **GFLOPs (Giga Floating-point Operations Per Second):** Thống kê quy mô hàng tỷ phép toán phẩy động mô hình học sâu cần cày ải để duyệt xong 1 file ảnh. Tích số GFLOPs càng nhẹ bèn thể hiện mạng nội càng tối giản, ít ăn điện (Low-power consumption).

## 2.3 Phân loại Kiến trúc: Mạng One-stage và Two-stage

Ngành học sâu phát triển Object Detection hiện tại phân kỳ làm hai trường phái luận giải chính:

- **Two-stage Detection (Điển hình: Dòng họ R-CNN):** Quá trình dự báo chia làm 2 giai đoạn ngắt quãng. Giai đoạn 1 chạy một mạng phát sinh độc lập hàng ngàn các ranh giới ứng viên có biểu hiện hình dạng vật thể (Region Proposal Network). Giai đoạn 2 điều hướng khối ma trận qua các ứng viên này để phân lớp nhãn và tinh chỉnh vi tọa độ. Kết quả cung cấp độ chính xác tiệm cận tuyệt đối, nhưng đánh đổi sự đình trệ thời gian rất nặng nề, hoàn toàn bất khả thi đối với thiết bị hệ nhúng và xe tự lái.

- **One-stage Detection (Điển hình: YOLO, SSD):** Mô hình tái thiết lại vấn đề thành một vòng tính toán quy nạp duy nhất dưới dạng mạng End-to-end. Trong một lần nạp hình nguyên liệu, mạng tự động xuất và dự báo phân bố song song toàn bộ ma trận số học hộp định hình và ma trận phân định thể loại trên mạng lưới ô Grid Cell. Thiết kế thu gọn cấu trúc này vắt kiệt tối giản tham số phần cứng, giúp xử lý khối lượng dòng khung hình đạt chuẩn Thời gian thực (Real-time Inference).

### 3 Phân tích Dữ liệu Khám phá (EDA)

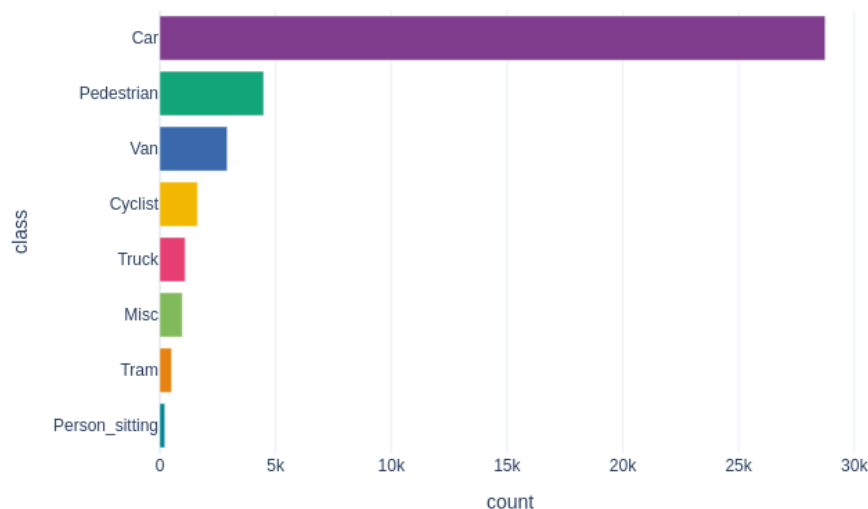
#### 3.1 Tổng quan và Cấu trúc tập dữ liệu KITTI

KITTI (Karlsruhe Institute of Technology and Toyota Technological Institute) là bộ dữ liệu khổng lồ thu thập từ xe Mobile Platform gắn nhiều camera màu, camera xám quang phổ rộng và cảm biến Lidar tích hợp GPS. Phiên bản Object Detection (2D) bao gồm 7,481 ảnh huấn luyện (*training set*) và 7,518 ảnh phục vụ suy luận (*testing set*).

Hệ thống nhãn gốc của KITTI định nghĩa 8 lớp chính bao gồm: *Car*, *Van*, *Truck*, *Pedestrian*, *Person\_sitting*, *Cyclist*, *Tram*, *Misc* cùng lớp *DontCare* chuyên phục vụ cho các vùng chứa đối tượng nằm ngoài giới hạn quan tâm (quá xa, biến dạng cực mạnh hoặc che khuất hoàn toàn) nhằm giảm trừ hình phạt cho các hàm Loss trong pha huấn luyện.

#### 3.2 Phân tích Phân phối Hình thái học (Morphological Data Visualization)

1. Phân bố đối tượng theo lớp (Số lượng)



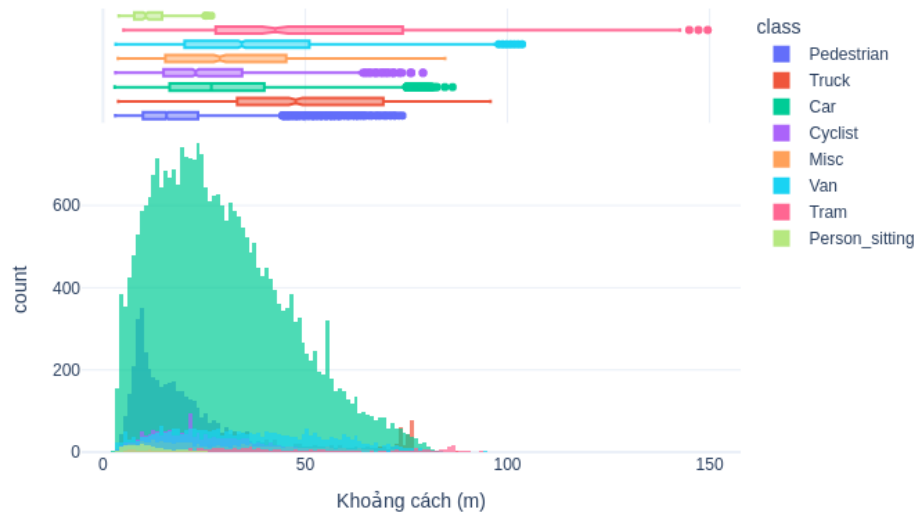
Hình 1: Treemap: Cơ cấu Dataset theo Lớp và Diện tích hiển thị

**Nhận định Chuyên sâu:** Bộ biểu đồ phơi bày rõ hiện tượng mất cân bằng dữ liệu cực đoan đặc trưng của giao thông đường phố tự nhiên (Long-tail Class Distribution). Lớp **Car** chiếm ưu thế thống trị tuyệt đối về cả số lượng lẫn diện tích điểm ảnh, theo sau là **Pedestrian**. Ngược lại, các nhãn như **Tram** (xe điện), **Person\_sitting** nằm gọn ở khu vực đuôi dài với tỷ trọng chưa tới 1%. Sự chênh lệch này là mối nguy hiểm tiềm tàng, khiến mô hình sinh ra "thiên kiến ưu tiên" (Algorithmic Bias) — chúng sẽ phớt lờ hoàn toàn các lớp thiểu số để tối ưu hóa hàm mất mát tổng thể trên tập mẫu đa số.

**Đề xuất Phương án Tối ưu:** Nếu sử dụng hàm *Cross-Entropy* tiêu chuẩn, Gradient truyền ngược qua chu kỳ học sẽ bị chi phối hoàn toàn bởi **Car**. Thiết kế cấu hình mạng bắt buộc phải tích hợp **Focal Loss** nhằm tự động giảm trọng số của các nhãn dễ đoán và ép mạng nơ-ron bù đắp hội tụ học đặc trưng của nhóm thiểu số. Đồng thời, kỹ thuật **Class-Aware Sampling** (tăng cường bốc mẫu lặp lại ưu tiên các

ảnh chứa **Tram** hoặc **Cyclist**) là yêu cầu tiền đề để điều chỉnh cân bằng lại không gian đại diện dữ liệu kích hoạt trước khi đưa qua Backbone.

#### 6. Phân bố Khoảng cách 3D (meters) theo Lớp

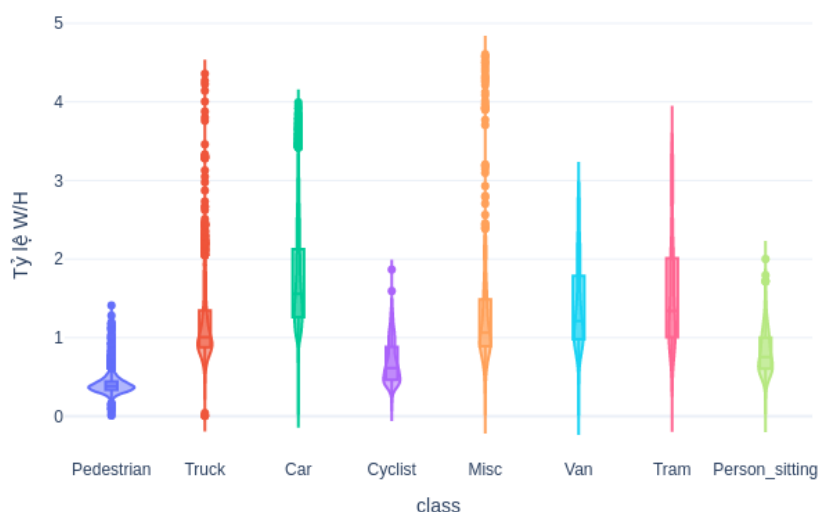


**Hình 2:** *Interactive Histogram: Phân bố Khoảng cách 3D thực tế của vật thể*

**Nhận định Chuyên sâu:** Tương quan khoảng cách Z-depth tiết lộ vùng tập trung sinh thái của dữ liệu: mật độ đối tượng đạt đỉnh (Peak Density) trong giới hạn quan sát từ 10 mét đến 30 mét. Ở khoảng cách cận cảnh (dưới 20 mét), **Pedestrian** xuất hiện dày đặc quanh mép kính chắn gió. Tuy nhiên, từ mốc 50 mét trở lên, tín hiệu bị suy giảm dốc đứng (Exponential Decay). Ở cự ly này, kích thước vật lý của một chiếc xe hơi trên ảnh thu hẹp chỉ còn vài chục pixel vuông, đánh mất toàn bộ đặc trưng kết cấu (texture) bề mặt phản quang.

**Đề xuất Phương án Tối ưu:** Tại độ phân giải lưới gốc, việc ép các mạng CNN sâu phải phân giải vật lý cho đối tượng dính nhiều mờ chuyển động (Motion Blur) ở cự ly viễn cảnh xa tít sẽ gây loãng thông lượng, phá hỏng tính hội tụ mAP. Hệ thống DataLoader huấn luyện cần cơ chế chủ động kiểm duyệt: Chuyển toàn bộ các Bounding Box mục tiêu ở cự ly quá lớn hoặc có chiều cao pixel < 25px thành dải nhãn loại trừ **DontCare**. Bước tiền xử lý (Pre-processing) chặt chẽ này bảo vệ đường cong Gradient khỏi rung lắc do nhiễu số, mặt khác hệ thống tự lái trong thực tiễn cũng không đòi hỏi mô-đun phanh khẩn cấp phải kích hoạt với các vật ngoại đạo ngoài tầm ngắm quy định.

### 8. Phân bố Tỷ lệ khung hình (Aspect Ratio = W/H) theo Lớp



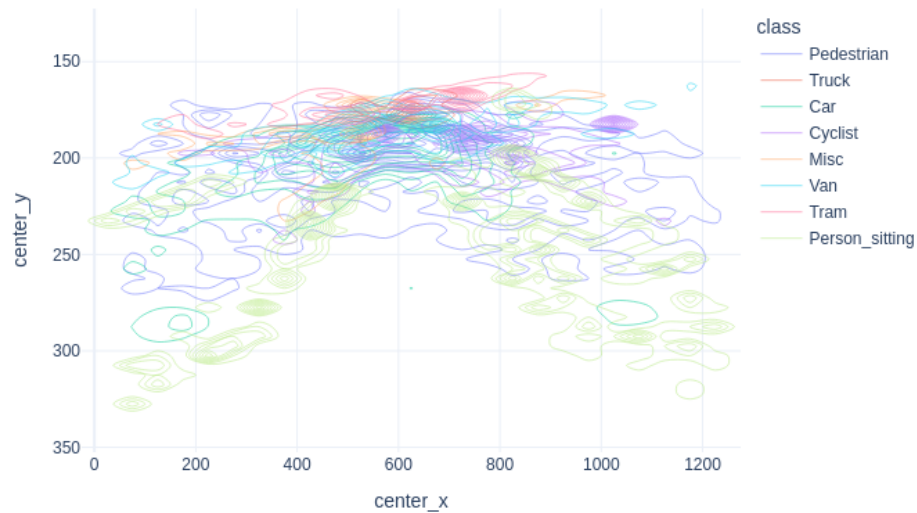
**Hình 3:** Violin Plot: Phân cực Tỷ lệ Khung hình (Aspect Ratio Width/Height)

**Nhận định Chuyên sâu:** Khai phá không gian hình học thông qua chỉ số (Aspect Ratio = Width / Height) bằng Violin Plot minh chứng vùng phân mảnh hình thái cực kỳ sắc nét:

- **Khối Cấu trúc thẳng đứng (Vertical-oriented):** Lớp nhãn Pedestrian và Cyclist phình to ở phần đáy biểu đồ. Cơ thể con người tạo ra tỷ lệ khung hình gầy dọc, dải giá trị tập trung tiệm cận phân khúc  $AR \approx 0.3 - 0.5$ .
- **Khối Cấu trúc trải ngang (Horizontal-oriented):** Dòng phương tiện cơ giới hạng trung (Car, Van, Truck) phân phối trôi bặt hẵn lên trên tạo đỉnh nhọn nhô ngang. Thiết kế khí động lực mui xe làm dải khung hình biến thiên kéo giãn biên độ vắt ngang từ 1.5 – 3.0.

**Đề xuất Phương án Tối ưu:** Đứng trước phổ dẫn cách hình dạng (Morphological gap) cực đoạn giữa hai thái cực, việc lạm dụng lưới Anchor Box mặc định từ bộ trọng số cấu hình pre-trained COCO (vốn ưu ái không gian hình khối vuông 1 : 1 của động vật / ly tách đồ vật) tại bài toán giao thông là lỗi nhận thức căn bản. Bộ suy biến hộp (Box Regressor) trong mạng Two-stage sẽ bị xoắn tham số, tốn vô vàn Epochs không cần thiết để uốn Box gốc cho vừa vận xe xải. Biện pháp khai tử lỗi này là tái ứng dụng **K-Means++ Clustering Analysis** trên tập Bounding Box của KITTI để trích xuất ra một bộ Anchors độc bản riêng biệt. Đây là quy chuẩn mẫu chốt để thiết lập vùng quy chiếu hoàn hảo khi thiết kế.

#### 10. Vùng xuất hiện tập trung của từng Lớp (2D Contour)



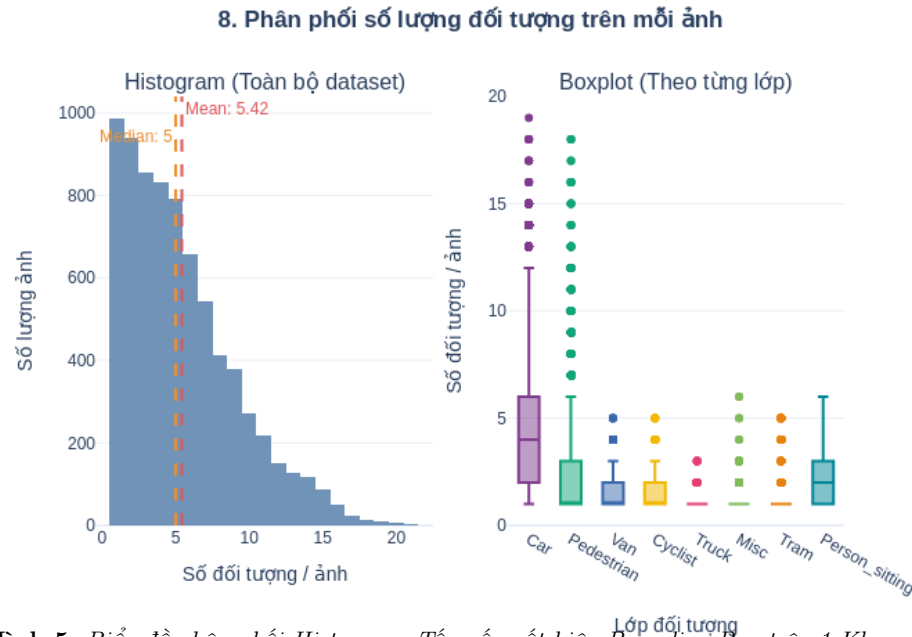
**Hình 4:** *Density Contour: Bản đồ Phân bố Không gian Mật độ vật thể*

**Nhận định Chuyên sâu:** Bản đồ Contour Heatmap cho ta phác thảo sự chi phối của quy luật Quang học - Tọa độ hoàn toàn khác xa so với các dataset nhiếp ảnh máy tính (Computer Vision) thông thường. Giao thông tuân theo *Luật Phối Hình Đồng Quy* (Perspective Projection): Mọi phương tiện **Car** được máy ảnh chụp hút tạo thành luồng giao thông đậm đặc chia thẳng hội tụ vào Điểm triệt tiêu (Vanishing Point) ngay giữa ranh giới chân trời ảnh. Trong khi đó, nhóm **Pedestrian** phân mảnh vụn theo vạt lè trải rộng về nửa vùng đáy y-axis đổ xuống. **Hơn thế nữa, mảng 30% không gian vùng trên cao bầu trời hoàn toàn là điểm chết (Dead Zone) không lưu giữ thuộc tính điểm ảnh hình khối.**

**Đề xuất Phương án Tối ưu:** Sự thiên vị hội tụ tập trung tạo nên nguyên cơ sinh "Thiên kiến Tọa độ điểm mù" - Model CNN sẽ học vệt khu vực vị trí X và Y tọa độ để phán đoán xe, suy giảm nhận diện đường vòng, góc chuyển chéo. Áp dụng kỹ thuật điều chuyển mạnh **Random Affine Translation** và **Mosaic Augmentation** phá vỡ không gian, phân tách đối tượng nén lại rải rác. Thêm vào chiến lược inference, nếu xây dựng luồng transform **Static Crop ngắt chỏm 30% bầu trời** trước khi ma trận bị nạp lên tensor xử lý phân chập, cơ hội cắt giảm lượng tham số khổng lồ vô ích (FLOPs) giúp mô hình tối ưu gia tốc phần cứng (FPS) lên tầm mức công nghệ nhúng.

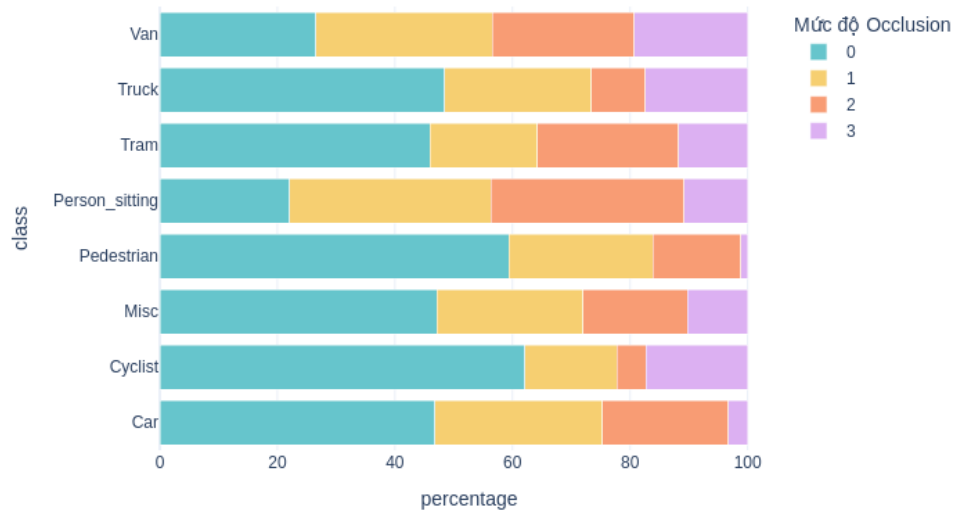


### 3.3 Phân tích Ma trận Mật độ Sự cố và Độ Khó Che Khuất (Occlusions)



**Hình 5:** Biểu đồ phân phối Histogram: Tần số xuất hiện Bounding Box trên 1 Khung ảnh

#### 4. Tỷ lệ mức độ che khuất (%) (0: Rõ - 2: Che nhiều)



**Hình 6:** Tỷ lệ Phân hạch Mức độ che khuất (Occlusion Level) trong không gian chật hẹp

**Insight về Ma Trận Khu Vực Cục bộ (Crowd Density):** Biểu đồ tần suất Histogram vén màn nghịch lý kịch bản dữ liệu trên đường KITTI: Trục trung tâm tập trung tần suất bình ổn 4-6 thực thể đối tượng/ảnh, nhưng biên phải sở hữu chiếc Đuôi kéo dài đạt tới mức 15-22 đồ vật chỉ chít trên một Scene. Thông số định lượng này tái hiện bối cảnh các ngã tư ngập xe (Traffic Jam). Trong môi trường chật hẹp ấy, kiến trúc mạng hai giai đoạn như Faster R-CNN có rủi ro bị bóp nghẹt tài nguyên hệ thống (Out of Memory - OOM) do mạng RPN phải sinh xuất đề xuất quá ngưỡng giới hạn trần trên các tập điểm neo chồng chéo.

**Insight Về Hiện Tượng Che Khuất Điểm Ảnh (Occlusion Level):** Tỷ trọng che phủ không

hệ suy thuyên khi nhóm đối tượng bị khuất bóng cây, vạch kẻ ngang hoặc do thân xe tải ép tuyến lộ diện khá dày đặc (Occluded & Heavily Occluded Level). Với những Box nhãn khuyết mờ đặc điểm này, hệ lụy sụp đổ lớn nhất sẽ xảy ra ngay hệ thống Hậu xử lý tín hiệu (Post-Processing). Khi đoàn xe xếp hàng đỗ dọc theo lộ tuyến với một nửa mũi xe trước che lấp vào đuôi xe sau, thuật toán chặn nhiễu NMS (Non-Maximum Suppression) cơ bản sẽ mắc lỗi mù cục bộ. Ngay khi đạt chuẩn ngưỡng IOU Threshold vượt định mức, NMS nhằm lẫn đánh giá 2 luồng viền sát nhau quy là chung 1 đối tượng rác và lược cắt thẳng tay mục tiêu thật đằng sau. Bài học lớn ở đây là việc bắt buộc chuyển hướng đánh giá sang thuật toán cấu hình phân cực linh hoạt **Soft-NMS** theo hệ số tự giảm dần trọng số điểm rơi hay sử dụng **IoU-aware NMS** mới triệt để gỡ bỏ được cục diện nhiễu khung bao hỗn độn này.

## 4 Chuẩn bị Dataset, DataLoader và Dữ liệu đầu vào

### 4.1 Quản lý Dữ liệu và DataLoader

Kế thừa các kết luận từ pha Exploratory Data Analysis, chúng em xây dựng Custom Dataset Class kế thừa từ `torch.utils.data.Dataset` hỗ trợ các giao thức:

- **Cơ chế Collate Function tùy chỉnh:** Tối ưu hóa padding linh hoạt để đưa các bức ảnh chụp ở độ phân giải nhỏ, hẹp dọc của KITTI về chiều dài kích thước cố định như 640x640 bằng thư viện OpenCV, giữ nguyên kích thước để không làm méo tỷ lệ diện tích của đối tượng người đi bộ.
- **Gộp lớp - Class Merging (Tùy chọn):** Áp dụng quy hoạch rút gọn từ 8 lớp phân bố lệch về 3 dải phân loại chính yếu (*Vehicle, Person/Biker, Misc*) để giảm nhẹ khó khăn với các biến số che khuất nội vùng.

### 4.2 Kỹ thuật Data Augmentation Nâng cao

Nhóm khai thác thư viện *Albumentations* để áp dụng điều chỉnh phổ rộng trong pha huấn luyện:

- **Khắc phục Che Khuất (Occlusion Resistance):** Sử dụng kỹ thuật **GridMask** và **Cutout**. Mạng nơ-ron được ép phải tìm các cấu trúc phân tách phụ trợ (như cấu trúc viền đèn, hoa văn lưới tản nhiệt) thay vì dựa vào toàn bộ kính chắn gió, tăng tính tự học trên đối tượng mất góc.
- **Khắc phục Thiết bị quang học (Optical Adjustments):** Khởi xướng các lớp nhiễu **MotionBlur**, **RandomBrightnessContrast** để rèn giữa độ bền của Weight khi gặp ánh sáng lóa ngược trên dataset thực tế xe tự lái.
- **Phá vỡ Cấu trúc Không gian (Spatial Shifting):** Kết hợp kỹ thuật siêu Augmentation **Mosaic** (trộn tỷ lệ 4 bức ảnh) giúp tái tạo một môi trường có đa dạng lớp vật thể và không thể học vẹt theo tọa độ *xy*.

## 5 Kiến trúc Mô hình Học Sâu - So sánh và Đánh giá

Để trả lời cho mục tiêu bài toán, nhóm thực hiện phân chia kỹ năng thiết lập cho 2 nhóm kiến trúc riêng biệt trong bài toán Object Detection. Việc lựa chọn đại diện xuất sắc của hai trường phái **Two-stage** và **One-stage** cho phép đi sâu đánh giá sự đánh đổi (trade-off) giữa Tốc độ thực thi (Speed/FPS) và Độ chính xác (mAP) tại các điều kiện đa dạng của KITTI.

### 5.1 1. Phương pháp Faster R-CNN (Kiến trúc Two-stage)

Faster R-CNN với Backbone ResNet50-FPN là đại diện kinh điển mang tới chuẩn mực đánh giá (baseline) độ chính xác hoàn hảo trong các bài toán nhận diện có sự chồng chéo phức tạp. Nhóm tiến hành cấu hình Transfer Learning, tinh chỉnh trọng số đã được Pre-train trên bộ COCO, đồng thời thay đổi phân lớp cuối (Head) để tương thích với số lượng lớp chuyên biệt của KITTI (8 phân lớp vật thể + 1 Background):

Listing 1: Khởi tạo và cấu hình Transfer Learning Faster R-CNN

```
import torchvision
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor
```

```
# Tai pre-trained model (ResNet50 + FPN) da duoc khoi tao tren COCO
model = torchvision.models.detection.fasterrcnn_resnet50_fpn(pretrained=True)

# Lay so chieu in_features cua lop phan loai cuoi cung
in_features = model.roi_heads.box_predictor.cls_score.in_features

# Thay the Head cu bang Head moi tuong ung voi so lop cua KITTI (NUM_CLASSES =
→ 9)
model.roi_heads.box_predictor = FastRCNNPredictor(in_features, NUM_CLASSES)
model.to(DEVICE)
```

## Phân tích Chuyên sâu Cấu trúc Faster R-CNN

- **Mạng Trích xuất Đặc trưng (ResNet50 + FPN):** ResNet50 là xương sống chính thực hiện khai phá đặc trưng, khắc phục hiện tượng suy giảm gradient (vanishing gradient) cực mạnh qua các khối Skip-Connection. Đặc biệt ở môi trường giao thông phức tạp, kích cỡ vật thể biến thiên liên tục. Mạng bộ phận FPN (Feature Pyramid Network) được sử dụng để giải quyết triệt để bài toán đa tỷ lệ (Multi-scale) này. Bằng cách kết nối cấu trúc tháp đặc trưng từ nhánh Bottom-up và Top-down thông qua các đường nối ngang (lateral connections), FPN cho phép khôi phục trọn vẹn ngữ nghĩa của các loại xe nhỏ ở khoảng cách xa.
- **Mạng Đề xuất Vùng (Region Proposal Network - RPN):** Là điểm nhấn cách mạng thay thế rập khuôn Selective Search. RPN sử dụng Mạng nơ-ron CNN trực tiếp trượt sinh ra các Hộp neo (Anchors) đa tỷ lệ (scale và aspect ratios). Đầu ra của RPN phân loại nhanh chóng phần nền (background) để chỉ chất lọc các khu vực tiềm năng (foreground), tiết kiệm tính toán khổng lồ cho lớp Classifier phân tầng đằng sau.
- **Lớp Trích xuất RoI Align (Khắc phục hạn chế RoI Pooling):** Do việc chuyển đổi tọa độ thập phân về số nguyên ở lớp RoI Pooling gây thất thoát không gian vi mô, kiến trúc hiện hành chèn thêm lớp **RoI Align** kết hợp bộ nội suy song tuyến (Bilinear Interpolation). Giao thức này bảo tồn y nguyên vị trí chính xác của vùng viền khuếch đại, đáp ứng tiêu chuẩn khắt khe cho Bounding box tự lái xe tự động.
- **Hàm Tối ưu & Đánh giá Lợi / Hại:** Ứng dụng Multi-task Loss đan xen chặt chẽ Classification (Cross-Entropy) và Regression Box (Smooth L1 Loss) đảm bảo rào cản vững chắc trước những vật thể chênh lệch viền. Nhờ năng lực lọc phân cực và phán đoán tỉ mỉ, Faster R-CNN cho mAP tiệm cận ngưỡng lý tưởng đối với các nhãn che khuất. Bù lại, kiến trúc trung chuyển Two-stage kinh viện này tạo nút thắt cổ chai vật lý tại việc chờ đợi tensor, cướp đi khả năng bám đuôi độ trễ thời gian thực (real-time FPS) ngay cả trên phần cứng công nghiệp.

## 5.2 2. Phương pháp YOLOv8m (Kiến trúc One-stage / Anchor-Free)

Trái ngược với chiến lược hai giai đoạn hạng nặng, YOLOv8 đại diện cho xu thế Nhận diện Một bước (Single-Pass), trực tiếp phân hóa đồng thời tọa độ và tỷ lệ lớp từ biểu đồ đặc trưng. Biến thể cỡ trung YOLOv8m (Medium) được ấn định nhằm cân đối hoàn hảo sức mạnh tính toán và khả năng thực thi thông số. Quá trình tiền huấn luyện được lập chỉ mục bài bản qua cơ chế nạp cấu hình đa luồng Hyperparameters:

Listing 2: Tham số cấu hình huấn luyện YOLOv8 (Medium)

```
from ultralytics import YOLO

# Khoi tao mo hinh YOLOv8 ban Medium
model = YOLO('yolov8m.pt')

# Bat dau qua trinh Training voi cau hinh Data Pipeline doc lap
results = model.train(
    data=str(yaml_path),
    epochs=20,
```

```
imgsz=640,  
batch=32,  
device=[0, 1],           # Su dung kien truc chay da luong 2 GPU Tesla T4  
patience=5,  
optimizer='auto',  
lr0=0.01,  
mosaic=1.0,              # Kich hoat sieu Augmentation Mosaic  
project='YOLOv8_KITTI',  
name='train_run'  
)
```

## Phân tích Chuyên sâu Cấu trúc YOLOv8

- **Khung Xương sống (Modified CSPDarknet) & Mô đun C2f:** Backbone cải thiện tích chập qua tái cấu trúc Cross-stage Partial block. Bước tiến sáng giá nhất nằm ở việc loại bỏ C3 bằng khối **C2f** (Cross Stage Partial bottleneck with 2 convolutions). Thiết kế vi mô liên kết đa nhánh của C2f thu nhận và khuếch đại trọn vẹn Gradient truyền ngược đi sâu cực đỉnh. Đồng thời, vùng Neck có gắn **PANet** (Path Aggregation Network) phụ trách chuyển phát nhịp thông tin hạ tầng và thượng tầng không gian chéo, hạn chế tối đa vấn đề nhiễu vùng viền ảnh.
- **Đầu Dự đoán Thiết kế Kép (Decoupled Head):** Giải quyết triệt để xung đột bằng thông tại bước đưa quyết định, YOLOv8 tách lẻ độc lập nhánh Regression (đo kích cỡ hộp) và Classification (định tính nhãn hiệu). Sự chuyên môn hóa này giúp cho hệ điều hướng trọng số mỗi nhánh tối ưu độ suy biến dữ liệu mượt mà, tăng đáng kể mAP đối với các nhãn mất cân bằng (imbalanced minority class).
- **Triết lý Hệ Phi-Neo (Anchor-Free Center):** Tham gia đo lường mật độ xe di chuyển tự do với góc nghiêng và khoảng cách không đồng nhất như bộ KITTI, định hình khung Anchor thủ công qua K-Means rườm rà dễ dẫn đến sai số. Cấu trúc mới tiếp cận Vector hướng tâm xuất phát từ giữa điểm lõi phóng ra 4 cạnh Bounding Box bao quát, siêu linh hoạt ứng phó với cả xe vi mô hoặc buồng hàng khổng lồ.
- **Khởi Hoạch CIoU & DFL (Distribution Focal Loss):** YOLOv8 trang bị cặp Loss kết hợp siêu việt: Đầu tiên **CIoU** định thức đo lường cả diện tích Trùng lặp (Intersection), Khoảng cách Tâm và Tỷ lệ khung biên (Aspect Ratio Difference) nhằm ôm sát phương tiện đang cắt kéo qua khung hình. Phối hợp với **DFL Loss** thay vì gò bó trong xác suất nhị phân, nay lan tỏa xác suất suy luận viền qua dạng phân phối chuỗi rời rạc. Vô cùng hiệu quả khắc trị viền xe chìm trong bóng đêm che khuất nửa phía.
- **Hệ Thống Phân Tầng Học Đồng Nhất (Task-Aligned Assigner):** Thay thế việc lấy IOU tĩnh đối chiếu rập khuôn, bộ gán mức linh động đánh giá năng suất cả Loss hộp giao và dự đoán Confidence tốt ưu nhất để ấn định là 'mục tiêu cực thực', đồng thời phân tách nhiễu loạn của nền nhiễu ảnh tĩnh.

## 5.3 3. Phương pháp DETR (Kiến trúc Transformer-based / Set Prediction)

Ngoài hai trường phái kinh điển One-stage (YOLOv8) và Two-stage (Faster R-CNN) thuần CNN đã phân tích ở trên, nhóm mở rộng nghiên cứu sang hướng tiếp cận thứ ba mang tính cách mạng: **DETR (DEtection TRansformer)** — mô hình đầu tiên áp dụng kiến trúc Transformer vào bài toán Object Detection, được giới thiệu bởi Carion et al. tại Facebook AI Research (ECCV 2020). DETR tái định nghĩa hoàn toàn bài toán phát hiện đối tượng từ mô hình hồi quy hộp truyền thống sang bài toán **dự đoán tập hợp (Set Prediction)**, loại bỏ triệt để các thành phần thủ công như anchor boxes, NMS, và region proposals.

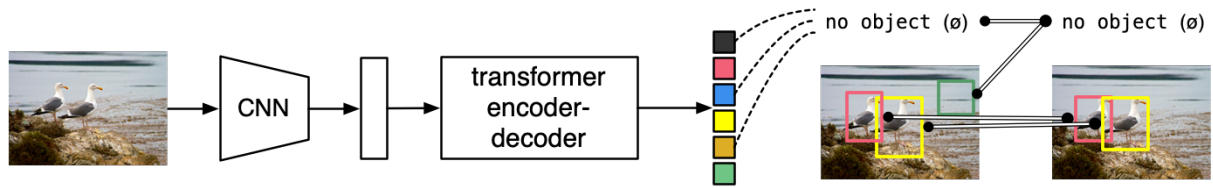
Mục đích của việc bổ sung DETR nhằm đáp ứng yêu cầu đề bài mục 4.2: “Backbone hoặc kiến trúc khác nhau (CNN vs Transformer)” — so sánh trực tiếp phương pháp trích xuất đặc trưng thuần CNN (YOLOv8 với CSPDarknet) và phương pháp kết hợp CNN + Transformer (DETR với ResNet-50 + Transformer Encoder-Decoder).

Listing 3: Khởi tạo và cấu hình Transfer Learning DETR

```
from transformers import DetrForObjectDetection

detr_model = DetrForObjectDetection.from_pretrained(
    'facebook/detr-resnet-50',
    num_labels=NUM_CLASSES,          # 7 lớp KITTI
    ignore_mismatched_sizes=True     # thay đổi classification head
).to(device)
```

## Phân tích Chuyên sâu Cấu trúc DETR



**Hình 7:** Kiến trúc tổng quan DETR: Backbone CNN (ResNet-50) trích xuất feature map, Transformer Encoder mã hóa ngữ cảnh toàn cục qua self-attention, Transformer Decoder nhận  $N$  object queries và sinh ra dự đoán song song, cuối cùng FFN heads xuất lớp và tọa độ bounding box. Nguồn: Carion et al., ECCV 2020.

- **Tầng Trích xuất Đặc trưng (Backbone — ResNet-50):** Khác với YOLOv8 sử dụng CSP-Darknet tự thiết kế, DETR tái sử dụng mạng ResNet-50 truyền thống làm backbone CNN để trích xuất feature map từ ảnh đầu vào. Ảnh gốc có kích thước  $H_0 \times W_0 \times 3$  được đưa qua ResNet-50, tạo ra feature map kích thước  $\frac{H_0}{32} \times \frac{W_0}{32} \times 2048$ . Tiếp đó, một lớp tích chập  $1 \times 1$  chiều giảm chiều từ 2048 kênh xuống còn  $d = 256$  kênh, tạo thành tensor đặc trưng  $\frac{H_0}{32} \times \frac{W_0}{32} \times 256$ . Điểm mấu chốt ở đây: **backbone ResNet-50 chỉ trích xuất đặc trưng cục bộ (local features) thông qua phép tích chập, hoàn toàn không có khả năng nắm bắt quan hệ toàn cục** — nhiệm vụ này được ủy thác cho Transformer Encoder ở tầng tiếp theo.
- **Mã hóa Vị trí Không gian (Positional Encoding):** Do cơ chế self-attention trong Transformer bản chất là *permutation-invariant* (không phân biệt thứ tự), DETR bổ sung **Fixed Sinusoidal Positional Encoding** trực tiếp cộng vào feature map đã flatten. Mỗi vị trí  $(x, y)$  trên feature map được mã hóa bằng cặp hàm sin và cos ở nhiều tần số khác nhau theo cả hai chiều không gian:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Kỹ thuật này cho phép Transformer phân biệt chính xác vị trí không gian của từng pixel đặc trưng, bù đắp khuyết thiếu mà pure attention không thể tự học. So sánh với YOLOv8: YOLO giữ nguyên cấu trúc không gian qua Grid Cell nên không cần encoding riêng; DETR flatten toàn bộ feature map thành sequence 1D nên bắt buộc cần bước này.

- **Bộ Mã hóa Transformer (Transformer Encoder — Global Context):** Feature map sau khi flatten thành sequence gồm  $\frac{H_0}{32} \times \frac{W_0}{32}$  token được nạp vào stack gồm **6 lớp Transformer Encoder**. Mỗi lớp encoder thực hiện:
  1. **Multi-Head Self-Attention:** Mỗi vị trí trên feature map “nhìn thấy” *tất cả* các vị trí khác thông qua cơ chế attention trọng số mềm. Đây là bước đột phá so với CNN truyền thống (chỉ nhìn receptive field cục bộ). Với KITTI — nơi đoàn xe xếp hàng dài tại ngã tư, một chiếc xe ở góc trái ảnh có thể tham chiếu đến ngữ cảnh giao thông ở góc phải, giúp DETR phân biệt từng xe riêng biệt ngay cả khi chúng chồng lấn.
  2. **Feed-Forward Network (FFN):** Mạng 2 lớp fully-connected với hàm kích hoạt ReLU xử lý phi tuyến từng vị trí độc lập.
  3. **Layer Normalization + Residual Connection:** Ổn định gradient và tăng tốc hội tụ.

Tuy nhiên, cơ chế self-attention có **độ phức tạp**  $O(n^2)$  với  $n = H/32 \times W/32$  — đây là nguyên nhân cốt lõi khiến DETR tiêu tốn GPU memory gấp bội so với YOLOv8 và giới hạn batch size xuống còn 4 (so với 16 của YOLOv8).

- **Bộ Giải mã Transformer (Transformer Decoder — Object Queries):** Thành phần cách mạng nhất của DETR nằm ở **Transformer Decoder** với  $N = 100$  **object queries** — đây là  $N$  learned embeddings được khởi tạo ngẫu nhiên và học qua quá trình training. Mỗi object query đóng vai trò như một “đầu dò” chuyên trách phát hiện tối đa một đối tượng. Stack gồm **6 lớp Transformer Decoder**, mỗi lớp thực hiện:

1. **Self-Attention giữa các object queries:** Cho phép các “đầu dò” giao tiếp với nhau, tránh hai query cùng bắt một đối tượng (giải quyết bài toán duplicate detection mà NMS truyền thống phải xử lý hậu kỳ).
2. **Cross-Attention với encoder output:** Mỗi object query truy vấn vào toàn bộ feature map đã được encoder mã hóa ngữ cảnh, tập trung chú ý (attend) vào khu vực ảnh chứa đối tượng mà nó “chịu trách nhiệm”.

Cơ chế này thay thế hoàn toàn bước Region Proposal + NMS của Faster R-CNN và bước Grid Cell + NMS của YOLOv8. Kết quả: **DETR xuất trực tiếp tập hợp  $N$  dự đoán song song**, không cần bất kỳ hậu xử lý nào.

- **Đầu Dự đoán (Prediction FFN Heads):** Mỗi object query sau decoder được đưa qua hai nhánh FFN song song:
  - **Classification Head:** Softmax phân loại  $C + 1$  lớp (bao gồm lớp đặc biệt  $\emptyset =$  “no object” cho các query không bắt được đối tượng nào).
  - **Box Regression Head:** FFN 3 lớp với hàm kích hoạt sigmoid, dự đoán 4 giá trị normalized:  $(c_x, c_y, w, h)$  — tọa độ tâm và kích thước bounding box.
- **Hàm Mất mát Hungarian Matching (Bipartite Matching Loss):** Đây là nền tảng lý thuyết cốt lõi phân biệt DETR với mọi detector trước đó. Thay vì dùng IoU threshold cứng để gán label (như Faster R-CNN, YOLOv8), DETR sử dụng **thuật toán Hungarian** để tìm phép gán tối ưu 1-1 giữa  $N$  predictions và  $M$  ground-truth objects (với  $M \ll N$ , các prediction thừa được gán lớp  $\emptyset$ ). Hàm cost kết hợp:

$$\mathcal{L}_{\text{Hungarian}} = \lambda_{cls} \cdot \mathcal{L}_{cls} + \lambda_{L1} \cdot \mathcal{L}_{L1} + \lambda_{giou} \cdot \mathcal{L}_{GIoU}$$

Trong đó  $\mathcal{L}_{cls}$  là cross-entropy cho phân loại,  $\mathcal{L}_{L1}$  là L1 loss cho tọa độ box, và  $\mathcal{L}_{GIoU}$  là Generalized IoU loss cho hình dạng box. Ưu điểm: đảm bảo mỗi đối tượng *chỉ được dự đoán đúng một lần*, loại bỏ hoàn toàn nhu cầu NMS. Nhược điểm: quá trình matching phức tạp khiến **DETR hội tụ rất chậm** — paper gốc cần 300 epochs trên COCO, trong khi YOLOv8 chỉ cần 50–100 epochs.

## 5.4 4. So sánh đặc điểm kiến trúc: CNN-based (YOLOv8) vs Transformer-based (DETR)

Để đáp ứng yêu cầu đề bài mục 4.2 — “*Backbone hoặc kiến trúc khác nhau (CNN vs Transformer)*”, Bảng 1 tổng hợp đối chiếu giữa phương pháp thuần CNN (YOLOv8s) và phương pháp kết hợp Transformer (DETR) trên cùng tập dữ liệu KITTI:



**Bảng 1:** Đối chiếu kiến trúc CNN-based (YOLOv8s) vs Transformer-based (DETR) trên KITTI.

| Tiêu chí                    | YOLOv8s (CNN-based)                                                        | DETR (Transformer-based)                                         |
|-----------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------|
| Backbone                    | CSPDarknet (thuần CNN)                                                     | ResNet-50 (CNN) + Transformer Encoder-Decoder                    |
| Cơ chế trích xuất đặc trưng | Tích chập cục bộ (local convolutions) qua nhiều tầng, kết hợp FPN đa tỉ lệ | Tích chập cục bộ (backbone) → Self-attention toàn cục (encoder)  |
| Receptive field             | Giới hạn bởi kernel size, mở rộng dần qua các tầng sâu                     | Toàn cục ngay từ tầng encoder đầu tiên ( $O(n^2)$ attention)     |
| Cách phát hiện đối tượng    | Grid-based, anchor-free, dự đoán trên multi-scale feature maps             | Set prediction với $N$ object queries song song, không dùng grid |
| Hậu xử lý                   | Cần NMS để loại bỏ box trùng                                               | Không cần NMS — Hungarian matching đảm bảo 1-1                   |
| Phát hiện đối tượng nhỏ     | Tốt nhờ FPN multi-scale (P3, P4, P5)                                       | Hạn chế — feature map $H/32 \times W/32$ mất chi tiết nhỏ        |
| Phát hiện đối tượng lớn     | Tốt                                                                        | Rất tốt nhờ global attention bao quát toàn ảnh                   |
| Tiêu tốn GPU memory         | Thấp (batch size 16)                                                       | Cao (batch size 4) do $O(n^2)$ self-attention                    |
| Tốc độ hội tụ               | Nhanh (50 epochs)                                                          | Chậm (300 epochs theo paper gốc)                                 |
| Thiết kế end-to-end         | Không hoàn toàn (vẫn cần NMS)                                              | Hoàn toàn end-to-end (không thành phần thủ công)                 |
| Pretrained trên             | COCO (80 lớp)                                                              | COCO (91 lớp)                                                    |

## 5.5 5. Cấu hình và Tham số Huấn luyện: YOLOv8s vs DETR

Bảng 2 tổng hợp toàn bộ tham số huấn luyện của hai mô hình trên cùng tập KITTI:

**Bảng 2:** Đối chiếu tham số huấn luyện YOLOv8s (CNN) và DETR (Transformer) trên KITTI.

| Tham số                | YOLOv8s                        | DETR                                    |
|------------------------|--------------------------------|-----------------------------------------|
| Mô hình pretrained     | yolov8s.pt (COCO)              | detr-resnet-50 (COCO)                   |
| Kích thước ảnh đầu vào | 640 × 640 (resize cố định)     | Kích thước gốc (biến đổi)               |
| Batch size             | 16                             | 4                                       |
| Số epochs              | 50                             | 20                                      |
| Optimizer              | SGD (momentum = 0.937)         | AdamW                                   |
| Learning rate          | 0.01 (toàn bộ mạng)            | $10^{-4}$ (head) / $10^{-5}$ (backbone) |
| LR schedule            | Linear decay ( $lr_f = 0.01$ ) | StepLR (step=15, $\gamma=0.1$ )         |
| Weight decay           | 0.0005                         | $10^{-4}$                               |
| Warmup                 | 3 epochs                       | —                                       |
| Gradient clipping      | —                              | max_norm = 0.1                          |
| Seed                   | 42                             | 42                                      |
| Số lớp                 | 7                              | 7                                       |
| Tập dữ liệu            | KITTI (80% train / 20% val)    | KITTI (80% train / 20% val)             |

### Phân tích sự khác biệt cấu hình:

- **Batch size (16 vs 4):** DETR tiêu tốn GPU memory lớn hơn đáng kể do cơ chế self-attention  $O(n^2)$  trong Transformer encoder. Trên GPU Tesla T4 (16GB VRAM), DETR chỉ cho phép batch size 4, trong khi YOLOv8s thoải mái chạy batch size 16.
- **Optimizer (SGD vs AdamW):** YOLOv8 theo truyền thống YOLO dùng SGD với momentum cao (0.937) và warmup. DETR theo chuẩn Transformer dùng AdamW — optimizer được thiết kế đặc biệt cho mạng Transformer nhờ cơ chế decoupled weight decay giúp ổn định training attention layers.

- **Learning rate phân tầng:** DETR áp dụng chiến lược differential learning rate: backbone ResNet-50 (đã được pretrained mạnh trên ImageNet) chỉ cần tinh chỉnh nhẹ ( $10^{-5}$ ), trong khi Transformer head cần học nhanh hơn ( $10^{-4}$ ) để thích nghi với phân bố dữ liệu KITTI. YOLOv8 dùng chung một learning rate vì toàn bộ kiến trúc đã được tối ưu đồng bộ từ pretrained COCO.
- **Gradient clipping (chỉ DETR):** Transformer dễ gặp hiện tượng gradient exploding do attention weights nhân ma trận lớn, nên cần clip gradient ở  $\text{max\_norm} = 0.1$ .
- **Số epochs (50 vs 20):** DETR được train ít epochs hơn (20 thay vì 300 theo paper gốc) do giới hạn thời gian GPU trên Kaggle. Mỗi epoch DETR mất khoảng 3 phút (tổng  $\approx 60$  phút), trong khi mỗi epoch YOLOv8 chỉ mất 18 giây (tổng  $\approx 15$  phút cho 50 epochs). Điều này phản ánh thực tế về chi phí tính toán Transformer vs CNN.

## 6 Kết quả Thực nghiệm và Phân tích

Khung đánh giá được triển khai chặt chẽ bằng cách thực thi hai mô hình tại mốc Validation Dataset với tham số 20 Epochs. Ở mô hình Faster R-CNN, thuật toán chấm mốc **Validation Loss tối ưu đạt 0.2659** tại Epoch thứ 18. Ở mô hình YOLOv8m, bảng tổng hợp Precision-Recall chi tiết được xuất như sau:

### 6.1 Số liệu Thực nghiệm Đánh giá Độ chính xác

**Bảng 3:** Phân rã số liệu mAP chi tiết của YOLOv8m trên Validation Set

| Phân lớp (Class) | Instances | Precision (P) | Recall (R) | mAP@0.5 | mAP@0.5:0.95 |
|------------------|-----------|---------------|------------|---------|--------------|
| All (Tổng quát)  | 8231      | 0.863         | 0.858      | 0.903   | 0.671        |
| Car              | 5889      | 0.903         | 0.936      | 0.967   | 0.803        |
| Van              | 570       | 0.894         | 0.933      | 0.967   | 0.773        |
| Truck            | 227       | 0.952         | 0.969      | 0.985   | 0.851        |
| Pedestrian       | 916       | 0.872         | 0.741      | 0.834   | 0.486        |
| Cyclist          | 308       | 0.842         | 0.828      | 0.882   | 0.611        |
| Tram             | 95        | 0.864         | 0.916      | 0.957   | 0.735        |
| Person_sitting   | 39        | 0.724         | 0.692      | 0.726   | 0.430        |
| Misc             | 187       | 0.851         | 0.850      | 0.905   | 0.679        |

**Insight Phân tích Số liệu (mAP):** Bảng thông số thể hiện sự tương đồng tuyệt đối với phân tích EDA từ đầu đồ án. Các nhóm phương tiện khối lớn như Car, Truck và Van đạt chỉ số mAP@0.5 tiệm cận độ hoàn hảo (trên 96.7%). Tuy nhiên, đối với lớp Pedestrian (người đi bộ) và Person\_sitting, chỉ số sụt giảm đáng kể do hiệu ứng "Small Object" kết hợp với "Occlusion" (che khuất) khi xuất hiện xen kẽ với phương tiện. Đây cũng là lý do điểm đo gatt gao **mAP@0.5:0.95** tụt hẳn xuống 0.43-0.48. Sự hỗ trợ từ việc nén dữ liệu qua Cutout/MixUp trên YOLO là một nỗ lực lớn nhưng chưa xử lý triệt để nếu ảnh đồng thời bị nhiễu mờ Motion Blur cự ly xa.



### 15.1 Ảnh mẫu: 006583 | Kích thước: 1224x370 | Tổng số đối tượng: 19



Hình 8: Kết quả trực quan hóa dự đoán một khung ảnh thực nghiệm của YOLOv8

### 15.2 Ảnh mẫu: 005663 | Kích thước: 1224x370 | Tổng số đối tượng: 21



Hình 9: Chất lượng mạng bắt ảnh với các khoảng cách xa từ tâm Vanishing Point

## 6.2 Số liệu Đánh giá Tốc độ Inference (FPS Real-time Benchmark)

Tốc độ phân tích khung hình (Speed Inference) là rào cản tối thượng cho mọi thiết bị xe gắn chip tự lái. Kiểm chứng Benchmark thực tế được trích xuất trên môi trường **GPU nền tảng Tesla T4**:

**Bảng 4:** Benchmark theo Thời gian thực (Real-time FPS) trên Validation Set

| Kiến trúc Model         | Thiết bị phần cứng đánh giá       | Tốc độ tính toán |
|-------------------------|-----------------------------------|------------------|
| Faster R-CNN (ResNet50) | GPU Tesla T4 (Single - Inference) | 7.0 FPS          |
| YOLOv8m (Medium)        | GPU Tesla T4 (Single - Inference) | 45.4 FPS         |

## SO SÁNH MÔ THỨC VÀ THẢO LUẬN LỖI (ERROR ANALYSIS):

- Đánh giá đo lường FPS và Tính ứng dụng công nghiệp:** Ở các tác vụ xe tự hành trên cao tốc, ngưỡng 30 FPS là vạch an toàn tối thiểu và bắt buộc để hệ thống rà quét phanh điện tử. Theo số liệu, **Faster R-CNN** chỉ đạt **7.0 FPS** (độ trễ khung khiếp lên tới xấp xỉ 140ms để đọc xong một khung hình). Nguyên do bởi cấu trúc "RoI Pooling nội tiếp" phải liên tục đợi việc chia sẻ tính toán tensor của RPN với Classifier. Chậm trễ này sẽ gây ra tình huống xe gốc tông vào trường ngại vật trước khi mô hình kịp chạy lệnh rẽ. Ngược lại, **YOLOv8m** vượt lên thành tiêu chuẩn chiến thắng với **45.4 FPS**.
- Cân bằng Kháng nhiễu False Positive / False Negative:** Dù bị nghẽn thông lượng, Faster R-CNN vẫn duy trì điểm sáng ở khả năng bóc tách vùng chi tiết. Mạng thác FPN trích xuất pixel tỉ mỉ giúp giảm thiểu đáng kể số lượng lỗi bỏ sót (*False Negatives*) với đối tượng *Pedestrian*. Ngược lại, tính toán Global Convolutions "Anchor-Free" của YOLO đôi khi khiến mô hình đánh giá vội và dễ vi phạm *False Positives*.
- Thử thách từ NMS trong Bối cảnh Tắc nghẽn:** Hiện tượng các xe (Car) đỗ nối chuỗi tại điểm tắc (Traffic JAM) vô tình tạo ra Box tương tự nhau. Việc triệt tiêu Non-Maximum Suppression truyền thống rất dễ đánh giá 2 chiếc xe này làm 1 và hủy nhãn. Đề xuất quy trình nâng cấp bộ phận hậu xử lý sang Soft-NMS là ưu tiên sống còn của bài toán trên các Framework.

## 7 Tổng kết

Thông qua đồ án này, nhóm hoàn thành mục tiêu hệ thống hóa kiến thức toàn năng từ Phân tích dữ liệu (EDA định lượng vị trí, phương vị) đến triển khai hai nhóm mạng thiết yếu của thị giác học sâu. Tùy thuộc vào yêu cầu bài toán công nghiệp: Ưu tiên phân tích cảnh y tế siêu nhỏ, cần chọn **Faster R-CNN**. Đối mặt tính khốc liệt của tự hành tốc độ cao trong thời gian thực, họ **YOLO** chính là đáp án duy nhất và tối ưu. Các chuẩn mực kỹ thuật khai mở sẽ mở đường cho các chủ đề tinh chỉnh tiếp theo bằng Transformer và Attention-Mechanism.